

Manual técnico

Instalación, configuración y mantención

Sistema de Gestión de Taller, Inventario y Ventas — Lubri-Express

Este manual es para quien instala y mantiene el sistema en el equipo del taller. El uso diario está en el *Manual de usuario*.

Índice

1. Qué es y qué necesita	2
2. Instalación en el equipo del taller	2
2.1. 1. PostgreSQL	2
2.2. 2. La aplicación	2
2.3. 3. El archivo .env	2
2.4. 4. El primer administrador	2
3. Migrar los datos del sistema anterior	3
3.1. Cómo leer el informe	3
3.2. Después de migrar: fijar los mínimos	3
4. Respaldos	3
4.1. Respaldar	3
4.2. Restaurar	4
4.3. Dejarlo automático	4
4.4. Una advertencia sobre las versiones	4
5. Empaquetar la aplicación	4
6. Correr desde el código fuente	4
7. Actualizar a una versión nueva	5
8. Problemas frecuentes	5
9. Verificación después de instalar	5

1 Qué es y qué necesita

Aplicación de escritorio en Python (PySide6) contra una base de datos PostgreSQL, ejecutándose **íntegramente en el equipo del taller**: no depende de internet y los datos no salen de ahí.

Componente	Versión
Windows	10 o superior (64 bits)
PostgreSQL	16 o superior
Python	3.10 o superior (solo si se corre desde el código fuente)
Espacio en disco	1 GB para la aplicación, más la base y los respaldos

El sistema se puede ejecutar de dos maneras: desde el **ejecutable** empaquetado (lo normal en el taller) o desde el **código fuente** (para desarrollo y para correr los scripts de migración y respaldo).

2 Instalación en el equipo del taller

2.1 1. PostgreSQL

Instalar PostgreSQL 16 desde <https://www.postgresql.org/download/windows/>. Durante la instalación se pide una contraseña para el usuario `postgres`: hay que anotarla, porque va en el archivo de configuración. Dejar marcada la instalación de las *Command Line Tools*: de ahí salen `pg_dump` y `psql`, que usan los respaldos.

Crear la base y cargar el esquema, desde el *SQL Shell (psql)* o desde `cmd`:

```
createdb -U postgres lubriexpress
psql -U postgres -d lubriexpress -v ON_ERROR_STOP=1 -f schema_lubriexpress.sql
```

El archivo `schema_lubriexpress.sql` está en `database/` del código fuente. Crea las tablas, las restricciones, los triggers de Kardex y la vista de stock crítico.

2.2 2. La aplicación

Copiar la carpeta `lubriexpress` (la que produce el empaquetado, sección 5) al equipo, por ejemplo en `C:\Lubri-Express`. Dentro va el ejecutable `lubriexpress.exe`.

2.3 3. El archivo .env

Junto al ejecutable debe quedar un archivo llamado `.env` con la dirección de la base:

```
DATABASE_URL=postgresql+psycopg2://postgres:LA_CONTRASENA@localhost:5432/lubriexpress
```

Sin ese archivo la aplicación no arranca, y lo dice con un cartel. Es a propósito: más vale que no abra a que se conecte en silencio a una base que no es la que se creía.

2.4 4. El primer administrador

Los usuarios se administran desde la aplicación, pero el primero hay que crearlo por consola, desde el código fuente:

```
.venv\Scripts\python scripts\crear_usuario.py
```

Pide nombre, usuario, rol y contraseña. Elegir rol **ADMINISTRADOR**. De ahí en adelante, todos los demás se crean desde la pestaña **Usuarios**.

Si alguna vez se pierde la contraseña del administrador, el mismo script la reemplaza: al escribir un usuario que ya existe, lo actualiza.

3 Migrar los datos del sistema anterior

Se hace una sola vez, antes de empezar a operar.

```
.venv\Scripts\python scripts\migrar_sistema_antiguo.py "C:\ruta\a\las\planillas" --
detalle
```

La carpeta debe tener las cinco planillas: `tabla clientes.xlsx`, `tabla vehiculo.xlsx`, `tabla productos.xlsx`, `tabla servicios.xlsx` y `tabla ordenes.xlsx`.

Todo entra en **una sola transacción**: o queda todo o no queda nada. El script se niega a correr dos veces (si ya existe el usuario `sistema_antiguo`, no hace nada), así que no hay forma de duplicar los datos por error.

3.1 Cómo leer el informe

Al terminar imprime cuánto cargó y, agrupado por motivo, cuánto descartó. Lo que hay que mirar:

- **Productos sin costo.** El sistema anterior tenía servicios anotados como productos (“Mano Obra”, “1/4 Hora taller”) con stock ficticio de miles de unidades. No se cargan. Con `-detalle` sale la lista: conviene revisarla y crear a mano los que sean servicios de verdad.
- **Stock negativo o fraccionario.** Se cargan con stock cero y hay que contarlos en la repisa (ajuste por recuento).
- **Patentes inválidas.** Autos extranjeros o datos mal escritos. Sus órdenes tampoco entran: no se les inventa un vehículo.
- **Homónimos fundidos.** El sistema anterior identificaba al cliente solo por su nombre, así que dos clientes con el mismo nombre quedan como uno. Si aparece uno, se separa a mano.

Al final verifica que el stock cuadre: lo que decían las planillas, lo que quedó en el Kardex y lo que quedó en los productos tienen que ser el mismo número. Si no cuadra, no confirma nada.

3.2 Después de migrar: fijar los mínimos

Los productos migrados quedan con stock mínimo cero, así que todo lo que esté en cero aparece como crítico. Se corrige en Inventario con **Mínimo por categoría**, una categoría a la vez.

4 Respaldos

4.1 Respaldar

```
.venv\Scripts\python scripts\respaldar.py
```

Deja un archivo comprimido en `respaldos\lubriexpress-AAAAMMDD-HHMM.sql.gz`, conserva los últimos 30 y borra los más viejos. Si en el `.env` está configurada la carpeta de OneDrive, deja además una copia ahí:

```
RESPALDO_ONEDRIVE=C:\Users\TU_USUARIO\OneDrive\Respaldos Lubri-Express
```

El cliente de OneDrive sube solo lo que cae en su carpeta, así que con copiar el archivo basta. Si la carpeta no existe, el script avisa y el respaldo local igual queda hecho.

`scripts\respaldar.py -listar` muestra lo que hay guardado.

4.2 Restaurar

```
.venv\Scripts\python scripts\restaurar.py respaldos\lubriexpress-20260916-2030.sql.gz
```

Pisa la base entera. Por eso pide escribir el nombre de la base para confirmar. Conviene probar una restauración al menos una vez, contra una base de prueba, antes de necesitarla de verdad.

4.3 Dejarlo automático

En el Programador de tareas de Windows, o con este comando en `cmd` como administrador (ajustando las rutas):

```
schtasks /create /tn "Respaldo Lubri-Express" /tr "C:\Lubri-Express\codigo\.venv\Scripts\python.exe C:\Lubri-Express\codigo\scripts\respaldar.py" /sc daily /st 20:00
```

A las 20:00, después del cierre. Conviene revisar cada tanto que los archivos estén apareciendo, tanto en `respaldos\` como en OneDrive.

4.4 Una advertencia sobre las versiones

El respaldo lo hace `pg_dump`, y **tiene que ser de la misma versión mayor que el servidor**: uno más nuevo escribe instrucciones que el servidor viejo no entiende al restaurar, y eso se descubre el peor día posible. El script lo verifica antes de escribir nada y se niega si no calzan. Si aparece ese mensaje, hay que instalar las herramientas cliente de la versión que corresponda.

5 Empaquetar la aplicación

El ejecutable **se construye en Windows**: PyInstaller no cruza de un sistema operativo a otro. Desde el código fuente, con el entorno virtual creado:

```
.venv\Scripts\pyinstaller.exe --noconfirm lubriexpress.spec
```

Queda en `dist\lubriexpress\`. Esa carpeta completa es lo que se copia al equipo del taller; el `.env` va adentro, al lado del `.exe`.

El archivo `lubriexpress.spec` incluye `qtbases.qm`, que es lo que hace que los botones de los diálogos digan “Guardar” y “Cancelar” en vez de “Save” y “Cancel”.

6 Correr desde el código fuente

Necesario para la migración, los respaldos y el empaquetado.

```
git clone https://github.com/CristobalEsp01/sist_lubriexpress.git
cd sist_lubriexpress
python -m venv .venv
.venv\Scripts\pip install -r requirements-dev.txt
copy .env.example .env
.venv\Scripts\python main.py
```

La batería de pruebas corre contra una base PostgreSQL real:

```
.venv\Scripts\python -m pytest
```

Si la base no está levantada, las pruebas que la necesitan se saltan solas en vez de fallar.

7 Actualizar a una versión nueva

1. Respalidar primero (`scripts\respaldar.py`).
2. Traer el código nuevo y volver a empaquetar.
3. Reemplazar la carpeta del ejecutable, **conservando el `.env`**.
4. Si el esquema de la base cambió, la nota de cambios lo dice y trae el bloque **ALTER** que hay que ejecutar; está en `docs/base-de-datos.md`.

8 Problemas frecuentes

“No se pudo conectar usando DATABASE_URL”

El servicio de PostgreSQL no está corriendo, o el `.env` no está junto al ejecutable, o la contraseña cambió. Verificar el servicio en *Servicios de Windows* y probar la conexión con `psql -U postgres -d lubriexpress`.

Los diálogos salen en inglés

Falta `qtbase_es.qm` dentro de `_internal\PySide6\Qt\translations\` en la carpeta empaquetada. Rehacer el empaquetado con el `.spec` del repositorio.

Los reportes o los PDF no aparecen donde se esperaba

Se guardan en la carpeta **Documentos del usuario que tiene la sesión de Windows abierta**, en `Documentos\Lubri-Express`. Si hay varias cuentas en el equipo, cada una tiene la suya.

El stock no cuadra con la repisa

Se corrige contando, con **Ajustar stock** (queda registrado en el Kardex). Nunca editando la base de datos a mano: el Kardex dejaría de explicar el stock, que es justamente lo que el sistema anterior no hacía.

9 Verificación después de instalar

1. El ejecutable abre y muestra la ventana de inicio de sesión.
2. Se entra con el administrador creado por consola.
3. Los botones de los diálogos están en español.
4. Las seis pestañas abren sin errores.
5. Una venta de prueba baja el stock y aparece en el Kardex.
6. Una orden de prueba se guarda y exporta su PDF.
7. `Documentos\Lubri-Express\` se crea al exportar.
8. `scripts\respaldar.py` deja un archivo y lo copia a OneDrive.
9. La tarea programada aparece en el Programador de tareas.
10. Al día siguiente, hay un respaldo nuevo.